Pineda Chavez Giovanni num. de cuenta: 323014302 |

Bitácora:

Sobrecarga contra sobrescritura:

Sobrecarga: Un ejemplo es el constructor de la clase Dispositivo. Existe Dispositivo(String id, int carga) y Dispositivo(String id). Ambos tienen el mismo nombre pero diferente cantidad de parámetros, permitiendo crear un dispositivo indicando la carga inicial o asumiendo un 100% por defecto.

Sobrescritura: Un ejemplo es el método Hay Alerta() en las hijas de Sensor. La clase Sensor lo define como abstracto, y clases como Sensor Temperatura lo sobrescriben (con @Override) para darle un comportamiento específico (evaluar si la temperatura supera la máxima).

Diferencia: La sobrecarga ocurre en la misma clase con distintos parámetros, mientras que la sobrescritura modifica el comportamiento de un método heredado en una clase hija manteniendo la misma firma.

Composición contra agregación:

La Batería es composición porque la clase Dispositivo hace el new Batería() dentro de su constructor. Si el dispositivo se destruye, la batería también muere con él; su ciclo de vida depende totalmente del creador.

Los sensores en la estación son agregación porque los objetos Sensor se crean fuera (en Main) y solo se insertan en el arreglo de la estación mediante el método instalar(). Como se vio en el ciclo 3, cuando el SensorPeso S2 fue retirado, no fue destruido; conservó su 78% de batería y pudo ser instalado nuevamente.

Modificadores:

Error al agregar báscula.gastar(5); en Main: java: gastar(int) has protected access in dispositivos.Dispositivo.

Explicación: El método gastar() está declarado como protected en la clase Dispositivo. Esto significa que solo puede ser invocado por las clases dentro del mismo paquete (dispositivos) o por sus clases hijas (como SensorPeso, que hereda de Sensor y este de Dispositivo). La clase Main se encuentra en un paquete distinto (estación) y no es hija de Dispositivo, por lo que el compilador bloquea el acceso.

Bravo Olmedo Paulo Cesar num. de cuenta:323297291

Bitácora:En esta práctica logramos aplicar con éxito el concepto de herencia para organizar nuestro código en varios paquetes (dispositivos, sensores, actuadores y estación), lo que nos evitó repetir líneas y nos permitió aprovechar las características de las clases padre.

Aprendimos a diferenciar claramente entre composición y agregación (como vimos con la batería y los sensores), a utilizar correctamente herramientas como super y this, y a entender por qué los modificadores de acceso como protected protegen los métodos para que solo las clases permitidas puedan usarlos. Al final, con las pruebas de ejecución en el Main y en la clase de temperatura, pudimos comprobar que todo funciona exactamente como esperábamos: los sensores miden, detectan alertas si rebasan los límites, y los actuadores responden de inmediato para controlar la estación de clasificación

Foto salida de Main

```
$ java -cp out estacion.Main
Instalado: Sensor S1 (100%) presencia
Instalado: Sensor S2 (80%) peso, limite 25.0
Instalado: Sensor S3 (100%) temperatura, max 45.0
Instalado: Actuador A1 (100%) banda, 2 m por ciclo
Instalado: Actuador A2 (100%) semaforo en apagado
Instalado: Actuador A3 (100%) alarma, volumen 10
== Ciclo 1 en Anden 3
S1 mide 12.0 cm
S2 mide 18.5 kg
S3 mide 40.0 grados
A1 avanza 2 m, lleva 2 m
A2 en verde
A3 en silencio
== Ciclo 2 en Anden 3
S1 mide 9.5 cm
S2 mide 32.0 kg ALERTA
S3 mide 42.0 grados
A1 se detiene
A2 en rojo
A3 suena a volumen 10
Retirado: S2
== Ciclo 3 en Anden 3
S1 mide 15.0 cm
S3 mide 46.5 grados ALERTA
A1 se detiene
A2 en rojo
A3 suena a volumen 10
Regresa: Sensor S2 (78%) peso, limite 25.0
Instalado: Sensor S2 (78%) peso, limite 25.0
A1 avanza 5 m, lleva 7 m
Anden 3: 3 sensores, 3 actuadores, 3 ciclos
```

Diagrama Completo:

